CE324-L Micro-Processor Interfacing

Project Proposal

Group Members:

Saad Mirza (2023498)

Moiz-ud-din (2023315)

Submitted to:

Engr. Abbas Khan

Faculty of Computer Science and Engineering

GIK Institute of Engineering Sciences and Technology

GHULAM ISHAQ KHAN INSTITUTE OF ENGINEERING SCIENCES & TECHNOLOGY

CE324 - Microprocessor Systems and Interfacing

AUTONOMOUS SURVEILLANCE ROVER

Multi-Sensor Human Detection with Real-Time Video Transmission

ABSTRACT

This project proposes the development of an autonomous surveillance rover designed for security and monitoring applications. The system integrates a Raspberry Pi 4B for high-level processing and computer vision tasks with an Arduino Uno for real-time motor control and sensor management. The rover features dual-factor human detection using a Pi Camera module and PIR motion sensor, autonomous obstacle avoidance, and wireless video transmission to a base station. Upon human detection, the system triggers audible and visual alarms while transmitting timestamped video footage and images to a laptop base station via WiFi. The rover is built on a 4-wheel RC car chassis and includes a manual override mechanism for emergency control. This distributed processing architecture ensures reliable real-time performance while maintaining sophisticated computer vision capabilities, making it suitable for perimeter security, warehouse monitoring, and search operations.

1. INTRODUCTION

1.1 Project Overview

The Autonomous Surveillance Rover is an intelligent mobile platform designed to patrol designated areas and detect human presence using multiple sensor modalities. The system combines the computational power of a Raspberry Pi 4B with the real-time responsiveness of an Arduino Uno microcontroller, creating a robust and reliable surveillance solution.

1.2 Motivation

Traditional surveillance systems are stationary and have limited coverage areas. Mobile surveillance platforms can extend monitoring capabilities, reduce blind spots, and provide dynamic responses to security events. This project addresses the need for an affordable, autonomous surveillance solution that can operate in various environments including warehouses, parking lots, and residential perimeters.

1.3 Key Features

- **Autonomous Navigation:** Self-directed movement with obstacle detection and avoidance
- **Dual-Factor Human Detection:** Camera-based computer vision combined with PIR motion sensing
- **Real-Time Alert System:** Immediate buzzer and LED activation upon detection
- **Video Documentation:** Captures and transmits before/after footage to base station
- **Manual Override:** RC-based emergency control for safety and testing
- **Wireless Communication:** WiFi-based data transmission to laptop base station

2. SYSTEM ARCHITECTURE

2.1 Hardware Architecture Overview

The system employs a distributed processing architecture where computationally intensive tasks are handled by the Raspberry Pi 4B, while time-critical control operations are managed by the Arduino Uno. This separation ensures reliable real-time performance for motor control and sensor polling while allowing sophisticated image processing and network communication.

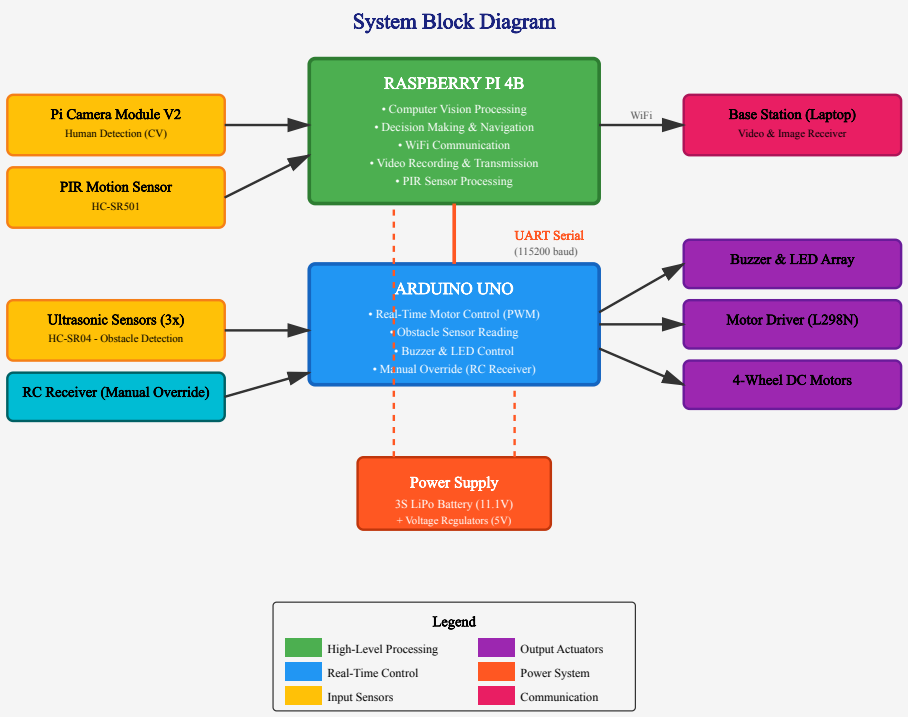

Figure 1: Complete System Block Diagram showing distributed processing architecture

2.2 Processing Distribution Rationale

The dual-processor architecture provides several critical advantages:

- **Real-Time Guarantee:** Arduino handles time-critical motor control without OS scheduling delays
- **System Reliability:** If Raspberry Pi crashes during intensive video processing, Arduino maintains obstacle avoidance
- **Parallel Processing:** Simultaneous execution of computer vision and motor control tasks
- **Emergency Response:** Arduino provides immediate response to manual override signals
- **Power Efficiency:** Offloading simple tasks to Arduino reduces Raspberry Pi workload

3. COMPONENT SPECIFICATIONS

3.1 Core Processing Units

Component	Specification	Quantity	Purpose
Raspberry Pi 4 Model B	4GB RAM, Quad-core ARM Cortex-A72, WiFi, 40-pin GPIO	1	Main controller, CV processing, communication
Arduino Uno R3	ATmega328P, 16MHz, 14 Digital I/O, 6 Analog inputs	1	Real-time motor control and sensor management
MicroSD Card	64GB Class 10 UHS-I	1	OS storage and video recording buffer

3.2 Mobility Platform

Component	Specification	Quantity	Purpose
RC Car Chassis	4-wheel, 1:10 scale, all-terrain wheels	1	Mobile platform base
DC Gear Motors	12V, 200 RPM with encoder (included in chassis)	4	Rover propulsion
Motor Driver	L298N Dual H-Bridge, 2A per channel	1	Motor speed and direction control

3.3 Sensing Systems

Component	Specification	Quantity	Purpose
Pi Camera Module V2	8MP, 1080p30, Sony IMX219 sensor	1	Human detection via computer vision
PIR Motion Sensor	HC-SR501, 7m range, 120° detection angle	1	Infrared motion detection (dual verification)
Ultrasonic Sensors	HC-SR04, 2cm-400cm range, 15° beam angle	3	Obstacle detection (front, left-front, right-front)

3.4 Alert and Communication Systems

Component	Specification	Quantity	Purpose
Active Buzzer	5V, 85dB at 10cm	1	Audible alert on human detection
LED Array	High-brightness Red LEDs, 5V with resistors	4	Visual alert indicator

RC Receiver	2.4GHz, 6-channel	1	Manual override control
WiFi Module	Built-in RPi 4B (802.11ac dual-band)	-	Base station communication

3.5 Power System

Component	Specification	Quantity	Purpose
LiPo Battery	3S (11.1V), 5000mAh, 30C discharge rate	1	Main power source
Voltage Regulator	LM2596 Buck Converter (5V, 3A output)	2	Power for RPi and Arduino
Power Distribution Board	PCB with fuse protection	1	Centralized power management

4. SYSTEM OPERATION

4.1 Operational Modes

4.1.1 Autonomous Patrol Mode

In this primary operating mode, the rover continuously navigates its environment while actively scanning for human presence:

1. **Navigation:** Raspberry Pi sends movement commands to Arduino via UART
2. **Obstacle Avoidance:** Arduino continuously polls ultrasonic sensors (50Hz) and adjusts motor speeds to avoid collisions
3. **Surveillance:** Pi Camera captures video stream at 30fps while RPi performs lightweight human detection using Haar Cascades or MobileNet SSD
4. **Motion Detection:** PIR sensor data processed by RPi for heat signature detection
5. **Path Planning:** Random walk with obstacle memory to maximize area coverage

4.1.2 Human Detection and Alert Sequence

When a potential human is detected, the following sequence executes:

1. **Primary Detection:** Computer vision identifies human silhouette in camera frame
2. **Verification:** System checks PIR sensor for concurrent infrared motion (dual-factor confirmation)
3. **If Both Sensors Confirm:**
    - Arduino triggers buzzer (3-second pulse pattern) and illuminates LED array
    - RPi captures 5 seconds of "before" video from buffer and 10 seconds "after" video
    - System captures high-resolution image of detection frame
    - Timestamped data packet transmitted to base station via WiFi socket
    - Rover holds position for 15 seconds before resuming patrol
4. **False Positive Handling:** If only one sensor triggers, log event but don't alert

4.1.3 Manual Override Mode

The RC receiver provides direct control when needed:

- Arduino monitors RC receiver signals with highest priority interrupt

- Any joystick movement immediately suspends autonomous control
- Direct PWM control of motors via RC input
- System returns to autonomous mode 5 seconds after last RC input
- Emergency stop via dedicated RC switch channel

4.2 Communication Protocol

4.2.1 Raspberry Pi ↔ Arduino Communication

Simple ASCII protocol over UART (115200 baud):

- `M:150,150` - Motor command (left speed, right speed: -255 to 255)
- `A:1` - Alert on (buzzer + LEDs)
- `A:0` - Alert off
- `S` - Request sensor status
- Arduino responds: `OBS:120,145,130` - Obstacle distances in cm (front, left, right)
- Arduino responds: `RC:1` - Manual override active

4.2.2 Rover ↔ Base Station Communication

TCP socket connection over WiFi for reliable data transmission:

- **Video Stream:** H.264 encoded stream at 720p, 15fps (bandwidth optimized)
- **Detection Packets:** JSON format containing timestamp, GPS coordinates (if available), detection confidence, sensor readings
- **Control Commands:** Base station can send patrol area boundaries, return-to-base commands

5. SOFTWARE ARCHITECTURE

5.1 Raspberry Pi Software Stack

- **Operating System:** Raspberry Pi OS Lite (64-bit)
- **Programming Language:** Python 3.9+
- **Computer Vision:** OpenCV 4.5+ with Haar Cascade or TensorFlow Lite with MobileNet SSD
- **Video Processing:** PiCamera library with H.264 hardware encoding
- **Communication:** PySerial (Arduino), Socket programming (base station)
- **Multithreading:** Python threading module for parallel task execution

5.2 Arduino Software

- **Development Environment:** Arduino IDE 2.0+
- **Programming Language:** C/C++
- **Libraries:** NewPing (ultrasonic), Servo (optional camera pan-tilt)
- **Real-Time Control Loop:** 20ms cycle time (50Hz)

5.3 Base Station Software

- **Platform:** Cross-platform (Windows/Linux/macOS)
- **Language:** Python with Tkinter/PyQt for GUI
- **Features:** Live video display, detection log, map view, alert notifications

6. PROJECT TIMELINE

Week	Phase	Milestones
Week 1	Component Procurement & Initial Setup	• Purchase all components • Set up Raspberry Pi OS and Arduino IDE • Test individual components (sensors, motors, camera) • Assemble 4-wheel RC car chassis • Install DC motors and test mechanical operation • Mount motor driver and verify motor control • Implement basic Arduino motor control code
Week 2	Arduino Integration - Sensors & Control	• Connect ultrasonic sensors to Arduino • Implement obstacle detection algorithm • Add buzzer and LED control • Integrate RC receiver for manual override • Test autonomous obstacle avoidance • Install OpenCV and required libraries • Establish UART communication with Arduino • Implement command protocol (M:, A:, S commands) • Test RPi-Arduino coordination
Week 3	Computer Vision Implementation	• Configure Pi Camera module • Implement human detection (Haar Cascade/MobileNet) • Optimize detection for real-time performance • Integrate PIR sensor with RPi • Implement dual-factor detection logic • Implement video buffering for "before" footage • Add video recording on detection event • Develop WiFi communication protocol • Create base station receiver software • Test video transmission reliability
Week 4	System Integration & Navigation	• Integrate all subsystems • Implement autonomous patrol algorithm • Fine-tune obstacle avoidance parameters • Optimize power consumption • Complete system testing • Extensive field testing in various environments • Debug edge cases and failure modes • Performance optimization • Prepare technical documentation • Record demonstration video
Week 5	Final Demonstration & Presentation	• Final system calibration • Live demonstration preparation • Presentation slides and poster • Project report finalization • **Final Demo Day**

Project Duration: 5 weeks (approximately 1 month)
Critical Path: Week 3 (Computer Vision) is the most technically challenging phase
Buffer Time: Week 4 provides contingency for unexpected delays

7. TESTING METHODOLOGY

7.1 Unit Testing

Component	Test Criteria
Ultrasonic Sensors	Measure accuracy at 20cm, 50cm, 100cm, 200cm distances. Verify 15° beam angle. Test response time < 50ms.
Motor Control	Verify PWM duty cycle 0-255 maps to speed. Test forward, reverse, turn left, turn right, stop. Measure response time < 20ms.
Pi Camera	Capture test images at various lighting conditions. Verify 1080p resolution. Measure frame rate ≥ 30fps.
PIR Sensor	Test detection range up to 7m. Verify 120° detection angle. Measure false positive rate in static environment.
WiFi Communication	Test connection stability at 5m, 10m, 20m ranges. Measure packet loss rate. Verify reconnection capability.

7.2 Integration Testing

- **RPi-Arduino Communication:** Send 1000 commands, verify 100% reception. Measure average latency < 10ms.
- **Obstacle Avoidance:** Navigate 10m course with 5 obstacles. Success = zero collisions.
- **Human Detection:** Test with 10 human subjects at distances 2m-8m, various angles. Target: >90% detection rate, <5% false positives.
- **Manual Override:** Verify immediate response (< 100ms) to RC input during autonomous operation.
- **Alert System:** Confirm buzzer activates within 200ms of detection. Verify LED visibility from 20m.

7.3 System Testing

- **Endurance Test:** Continuous operation for 60 minutes. Monitor battery voltage, CPU temperature, memory usage.
- **Environmental Testing:** Operate in indoor, outdoor, low-light, and high-light conditions.
- **Video Transmission:** Verify complete video file reception at base station. Check timestamp accuracy.
- **Multi-Human Scenario:** Test detection with 1, 2, and 3 humans simultaneously in frame.
- **Edge Cases:** Test behavior during WiFi disconnection, battery low voltage, sensor failure.

7.4 Demonstration Plan

> **Live Demonstration Sequence:**
>
> 1. Power on rover and base station - show system initialization
> 2. Demonstrate manual override control via RC transmitter
> 3. Switch to autonomous mode - rover navigates obstacle course

4. Team member walks into rover's field of view
5. Show dual-factor detection (camera + PIR confirmation)
6. Display audible/visual alerts (buzzer + LEDs)
7. Show real-time video transmission to base station laptop
8. Display received before/after footage and detection image

Backup Plan: Pre-recorded demonstration video showing all features in various environments

8. EXPECTED CHALLENGES & MITIGATION

Challenge	Impact	Mitigation Strategy
False positives in human detection	Unnecessary alerts reduce system credibility	Dual-factor verification (camera + PIR). Adjustable confidence threshold. Training with diverse dataset.
WiFi range limitations	Video transmission fails at distance	Use 5GHz band for less interference. Buffer video locally. Implement automatic reconnection.
Real-time processing load	System lag or crashes	Use lightweight CV models (MobileNet). Reduce video resolution to 720p. Offload motor control to Arduino.
Power consumption	Limited operation time	5000mAh battery provides ~2 hours. Implement low-power mode. Option to add second battery.
Rough terrain navigation	Rover gets stuck or tips over	Use all-terrain RC car with high ground clearance. Implement tilt detection. Test on various surfaces.
Camera focus in varying light	Poor detection accuracy	Use automatic exposure. Add IR LEDs for night vision. Tune CV parameters for environment.

9. FUTURE ENHANCEMENTS

If time and resources permit, the following features can be integrated:

Enhancement	Description	Additional Components
GPS Tracking	Log patrol path and detection locations for mapping	Neo-6M GPS module
Night Vision	Operate in complete darkness using infrared illumination	Pi NoIR camera + IR LED array
Pan-Tilt Camera	Wider surveillance area without rover movement	2x servo motors + mounting bracket

Gas Sensors	Detect smoke, CO2, or other hazardous gases	MQ-2/MQ-135 gas sensors
Mobile App Control	Control rover and view video stream from smartphone	Software development only
Facial Recognition	Identify specific individuals vs. intruders	Software + training database
Solar Charging	Extended operation time with renewable energy	6V solar panel + charge controller
Multiple Rover Coordination	Deploy multiple units for larger area coverage	Additional rover units

10. LEARNING OUTCOMES

This project provides hands-on experience with multiple aspects of embedded systems:

- **Microprocessor Interfacing:** GPIO programming, UART, PWM, I2C protocols
- **Real-Time Systems:** Understanding timing constraints, interrupt handling, task scheduling
- **Computer Vision:** Image processing, object detection algorithms, performance optimization
- **Control Systems:** Motor control, PID tuning, sensor fusion
- **Wireless Communication:** WiFi protocols, socket programming, data streaming
- **Power Management:** Voltage regulation, battery management, power consumption analysis
- **System Integration:** Combining multiple subsystems into cohesive platform
- **Embedded Linux:** Raspberry Pi OS configuration, Python programming, cross-compilation
- **Hardware-Software Co-design:** Distributed processing, task allocation optimization

11. SAFETY CONSIDERATIONS

Electrical Safety:

- All circuits protected with appropriate fuses
- LiPo battery handled with proper charging and storage protocols
- Voltage regulators with thermal protection
- No exposed high-voltage connections

Operational Safety:

- Emergency stop via RC transmitter dedicated channel
- Speed limited to safe indoor/outdoor operation (< 2 m/s)
- Obstacle detection prevents collisions with people or objects
- Manual override always takes precedence over autonomous control

Privacy Considerations:

- Clear indication (LED/buzzer) when recording is active
- Video data encrypted during transmission
- Compliance with local surveillance regulations

12. CONCLUSION

The Autonomous Surveillance Rover project represents a comprehensive application of microprocessor systems and interfacing concepts learned in CE324. By combining the Raspberry Pi 4B's computational capabilities with Arduino Uno's real-time control, we create a robust, reliable, and sophisticated surveillance platform.

The dual-factor human detection system (camera + PIR) demonstrates understanding of sensor fusion and verification techniques critical for real-world applications. The distributed processing architecture showcases efficient task allocation between high-level and low-level controllers, a key concept in modern embedded systems design.

This project goes beyond theoretical knowledge by implementing practical features such as wireless video transmission, autonomous navigation, and manual override—all essential for a deployable surveillance system. The modular design allows for future enhancements and scalability, making it an excellent learning platform for embedded systems development.

We are confident that this project will not only meet the CE324 course requirements but also demonstrate practical engineering skills and innovative thinking. The combination of hardware integration, real-time software, computer vision, and wireless communication makes this a standout project that showcases comprehensive understanding of microprocessor systems.

13. REFERENCES

1. Raspberry Pi Foundation. (2024). *Raspberry Pi 4 Model B Datasheet*. Retrieved from https://www.raspberrypi.org
2. Arduino. (2024). *Arduino Uno R3 Technical Specifications*. Retrieved from https://www.arduino.cc
3. Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.
4. Viola, P., & Jones, M. (2001). "Rapid Object Detection using a Boosted Cascade of Simple Features." *IEEE Conference on Computer Vision and Pattern Recognition*.
5. Howard, A. G., et al. (2017). "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." *arXiv:1704.04861*.
6. STMicroelectronics. (2024). *L298N Dual H-Bridge Motor Driver Datasheet*.
7. Monk, S. (2016). *Programming the Raspberry Pi: Getting Started with Python*. McGraw-Hill Education.